

Notebook

May 12, 2026

0.1 Strava Data Analysis: Yosemite Half Marathon

On May 9th, 2026, my fiancé and I ran the Vacation Race's Yosemite Half Marathon, and we had a blast! The training was... less fun. We are extremely amateur athletes; this event marked my second half marathon and my fiancé's first. However, I love running, and my fiancé loves me, so the 12-week training program commenced on February 16th, 2026.

This notebook will provide insight into my actual training, using the Strava API. As an avid Strava user since 2018 (I was a sophomore in high school!), all my running-related data lives somewhere in their application. I figured that it's time to make use of what's there!

```
[ ]: import marimo as mo
import os
from dotenv import load_dotenv
import requests
import json
from datetime import datetime, timezone
import time
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from matplotlib.patches import Patch
import numpy as np

load_dotenv()

CLIENT_ID = os.getenv("STRAVA_CLIENT_ID")
CLIENT_SECRET = os.getenv("STRAVA_CLIENT_SECRET")
REFRESH_TOKEN = os.getenv("STRAVA_REFRESH_TOKEN")

if not CLIENT_ID or not CLIENT_SECRET or not REFRESH_TOKEN:
    raise ValueError("Missing credentials for Strava API.")
```

```
[ ]: def get_access_token(client_id, client_secret, refresh_token):
    resp = requests.post(
        "https://www.strava.com/oauth/token",
        data={
            "client_id": client_id,
            "client_secret": client_secret,
```

```

        "refresh_token": refresh_token,
        "grant_type": "refresh_token",
    },
)
resp.raise_for_status()
return resp.json()["access_token"]

```

```
ACCESS_TOKEN = get_access_token(CLIENT_ID, CLIENT_SECRET, REFRESH_TOKEN)
```

```

[ ]: CACHE_PATH = "strava_data.json"
DATE_FROM = datetime(2026, 2, 16, tzinfo=timezone.utc)
DATE_TO = datetime(2026, 5, 10, tzinfo=timezone.utc)
CACHE_KEY = f"{DATE_FROM.date()}_{DATE_TO.date()}"

```

0.2 Let's get some data!

I want to fetch all the runs from my 12-week training program, which started on February 16, 2026 and ended on May 9, 2026 (race day!).

I've written this to respect the rate limit of the Strava API: 100 calls every 15 minutes. Although I don't think I did >8 runs a week, it's best to bake this in anyway.

Finally, the API should only get called once— the first time you run this script. The returned data will get cached in JSON format. This cell will check if data has already been ingested before making any requests, avoiding any unnecessary calls. As one of my undergrad writing professors always said: “Do it nice, or do it twice.”

```

[ ]: def fetch_runs(access_token, date_from, date_to):
    after_ts = int(date_from.timestamp())
    before_ts = int(date_to.timestamp())
    headers = {"Authorization": f"Bearer {access_token}"}
    activities, page = [], 1

    while True:
        resp = requests.get(
            "https://www.strava.com/api/v3/athlete/activities",
            headers=headers,
            params={"after": after_ts, "before": before_ts, "per_page": 100,
↪ "page": page},
        )

        if resp.status_code == 429:
            retry_after = int(resp.headers.get("X-RateLimit-Reset", 60))
            time.sleep(retry_after)
            continue

        resp.raise_for_status()
        time.sleep(1)

```

```

        batch = resp.json()
        if not batch:
            break

        activities.extend(batch)
        page += 1

    return [a for a in activities if a.get("sport_type") == "Run"]

cache = {}
if os.path.exists(CACHE_PATH):
    with open(CACHE_PATH) as f:
        cache = json.load(f)

if CACHE_KEY in cache:
    print(f"loaded {len(cache[CACHE_KEY])} runs from cache")
    raw_runs = cache[CACHE_KEY]
else:
    print("fetching runs from Strava API...")
    raw_runs = fetch_runs(ACCESS_TOKEN, DATE_FROM, DATE_TO)
    cache[CACHE_KEY] = raw_runs
    with open(CACHE_PATH, "w") as f:
        json.dump(cache, f, indent=2)
    print(f"{len(raw_runs)} runs fetched and cached")

```

loaded 30 runs from cache

Now that I've gotten ahold of my data, I've picked out some features that may be relevant to my analysis. In my persistent fight against using the Imperial System, I've converted all distance measurements to km-based units.

```

[ ]: def build_df(runs):
    rows = []
    for a in runs:
        rows.append({
            "id": a["id"],
            "name": a["name"],
            "date": pd.to_datetime(a["start_date_local"]),
            "distance_km": round(a["distance"] / 1000, 2),
            "moving_time_min": round(a["moving_time"] / 60, 1),
            "elapsed_time_min": round(a["elapsed_time"] / 60, 1),
            "elevation_m": a.get("total_elevation_gain", 0),
            "avg_hr": a.get("average_heartrate"),
            "max_hr": a.get("max_heartrate"),
            "avg_speed_kmh": round(a["average_speed"] * 3.6, 2),
            "suffer_score": a.get("suffer_score"),
            "workout_type": a.get("workout_type"),

```

```

        "trainer": a.get("trainer", False),
        "achievement_count": a.get("achievement_count"),
        "kudos_count": a.get("kudos_count")
    })

df = pd.DataFrame(rows).sort_values("date").reset_index(drop=True)
df["pace_min_per_km"] = df["moving_time_min"] / df["distance_km"]
df["week"] = df["date"].dt.to_period("W")
df["week_num"] = (df["week"] - df["week"].min()).apply(lambda x: x.n) + 1
return df

df = build_df(raw_runs)
df.head()

```

/var/folders/t4/dt44t_wj0ssdz_09tnsh52h00000gn/T/marimo_18444/__marimo__cell_iLI
n.py:24: UserWarning: Converting to PeriodArray/Index representation will drop
timezone information.

```
df["week"] = df["date"].dt.to_period("W")
```

Q Search...

id int64	name str	date datetime64[us, UTC]	distance_km float64	moving_time_min float64	elapsed_time_min float64
17,412,349,146	Evening Run	2026-02-15 19:24:05+00:00	4.84	29.9	
17,422,836,430	Afternoon Run	2026-02-16 16:44:48+00:00	9.67	66.0	
17,457,179,911	Evening Run	2026-02-19 18:33:12+00:00	6.45	45.2	
17,511,622,117	Lunch Run	2026-02-23 11:35:12+00:00	11.28	83.2	
17,582,941,801	Afternoon Run	2026-03-02 17:07:36+00:00	4.35	30.0	

5 rows, 18 columns

No selection

0.3 Weekly Load

With my features in order, the first thing I want to look at is how much I was *doing* each week. A few significant family emergencies happened in April, so I expect there to be some variation in my training load around those weeks.

According to [Coros](#), an optimal half marathon training plan should build mileage at a rate of around 10% per week, with long runs making up $\leq 30\%$ of the total mileage. My plan included a taper that began approximately a week and a half before race day, so the load should drop off starting in Weeks 10-11.

In terms of data visualization, this should look like a steady upward trend in load for the first 9-10 weeks, followed by a drop leading into race day.

```
[ ]: weekly = (
    df.groupby("week_num")
```

```

.agg(
    runs=("id", "count"),
    total_km=("distance_km", "sum"),
    longest_km=("distance_km", "max"),
    avg_pace=("pace_min_per_km", "mean"),
    avg_hr=("avg_hr", "mean"),
    elevation_m=("elevation_m", "sum"),
    avg_suffer_score=("suffer_score", "mean"),
    total_suffering=("suffer_score", "sum")
)
.round(2)
.reset_index()
)

mo.ui.table(weekly)

```

Table Bar Chart X +

Q Search...

☐	week_num int64	runs int64	total_km float64	longest_km float64	avg_pace float64	avg_hr float64	elevat float64
☐	1	1	4.84	4.84	6.18	170.90	
☐	2	2	16.12	9.67	6.92	155.20	
☐	3	1	11.28	11.28	7.38	155.60	
☐	4	4	28.35	10.38	6.52	164.62	
☐	5	3	25.72	12.56	6.26	165.23	
☐	6	3	26.18	13.94	6.25	164.20	
☐	7	3	19.73	11.52	6.15	164.37	
☐	8	3	32.71	17.71	5.89	170.73	
☐	9	2	11.29	7.49	6.32	157.30	
☐	10	3	27.93	21.09	6.64	158.87	
☐	11	1	9.69	9.69	6.19	167.80	
☐	12	3	23.90	11.88	5.84	169.23	
☐	13	1	21.05	21.05	5.64	178.40	

13 rows, 9 columns No selection

```

[ ]: fig1, ax1 = plt.subplots(figsize=(10, 4))

ax1.bar(
    weekly["week_num"], weekly["total_km"],
    color="#4A90D9", alpha=0.85, width=0.6, label="Weekly km",
)

```

```

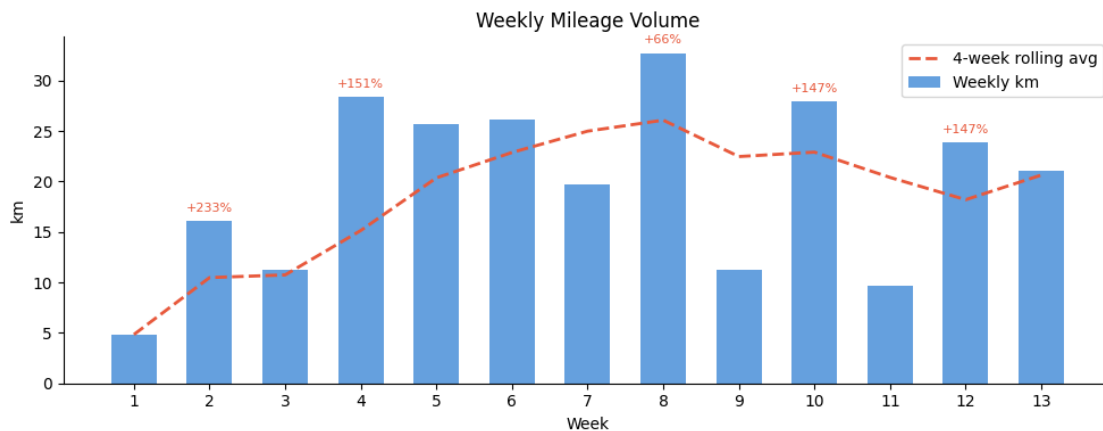
rolling_km = weekly["total_km"].rolling(4, min_periods=1).mean()
pct_change_km = weekly["total_km"].pct_change()

ax1.plot(weekly["week_num"], rolling_km, color="#E8593C", linewidth=2,
        linestyle="--", label="4-week rolling avg")

for i, (w, pct_km) in enumerate(zip(weekly["week_num"], pct_change_km)):
    if pct_km > 0.10:
        ax1.annotate(
            f"+{pct_km*100:.0f}%",
            xy=(w, weekly["total_km"].iloc[i]),
            xytext=(0, 6), textcoords="offset points",
            ha="center", fontsize=8, color="#E8593C",
        )

ax1.set_xlabel("Week")
ax1.set_ylabel("km")
ax1.set_title("Weekly Mileage Volume")
ax1.xaxis.set_major_locator(ticker.MultipleLocator(1))
ax1.legend()
ax1.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.gca()

```



Even with a few (expected) funky weeks, the training plan looks solid. However, mileage doesn't tell the whole story. I want to take a look at how difficult these runs were physiologically. Here's where Strava's "Suffer Score"—now called "Relative Effort"—comes in. Traditionally, I'd approach this kind of analysis by using max heart rate as a proxy for the strain of a given training session, then aggregate those values over each week of my plan. Relative Effort does this for me, accounting for the accumulation of effort over time and increased fitness during the training regimen. Let's see how my suffering stacks up over these 12 weeks. I'm going to call this metric my "Total Suffering".

0.4 Suffer Score / Relative Effort

```
[ ]: fig2, ax2 = plt.subplots(figsize=(10, 4))

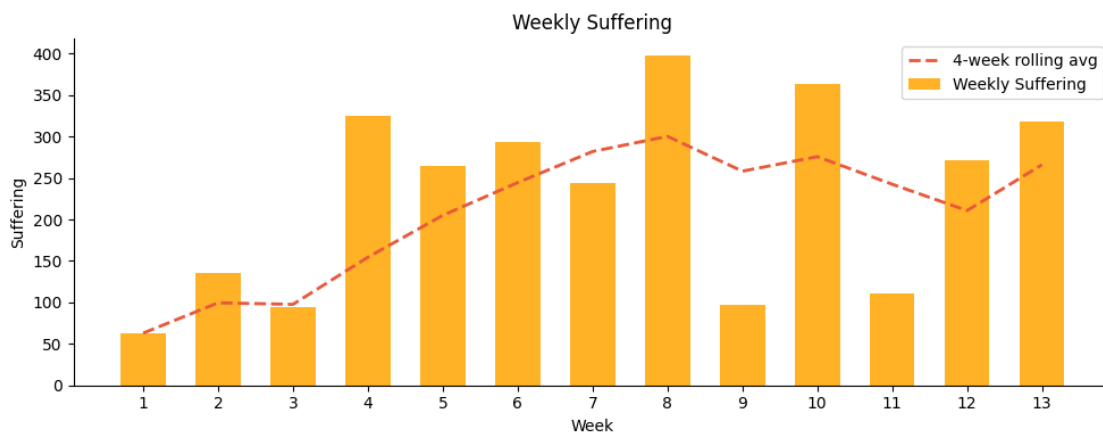
ax2.bar(
    weekly["week_num"], weekly["total_suffering"],
    color="#FFA500", alpha=0.85, width=0.6, label="Weekly Suffering",
)

rolling_suf = weekly["total_suffering"].rolling(4, min_periods=1).mean()
pct_change_suf = weekly["total_suffering"].pct_change()

ax2.plot(weekly["week_num"], rolling_suf, color="#E8593C", linewidth=2,
        linestyle="--", label="4-week rolling avg")

for j, (w2, pct) in enumerate(zip(weekly["week_num"], pct_change_suf)):
    if pct > 0.10:
        ax1.annotate(
            f"+{pct*100:.0f}%",
            xy=(w2, weekly["total_suffering"].iloc[j]),
            xytext=(0, 6), textcoords="offset points",
            ha="center", fontsize=8, color="#E8593C",
        )

ax2.set_xlabel("Week")
ax2.set_ylabel("Suffering")
ax2.set_title("Weekly Suffering")
ax2.xaxis.set_major_locator(ticker.MultipleLocator(1))
ax2.legend()
ax2.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.gca()
```



These look tightly coupled, which is expected. Let's look at the correlation between some of these variables and determine where the biggest variance between total distance and total suffering comes in.

```
[ ]: suffer_df = df.dropna(subset=["suffer_score", "max_hr", "avg_hr"]).copy()

# Combo index: z-score each predictor and sum them
for col in ["distance_km", "max_hr", "elevation_m"]:
    suffer_df[f"z_{col}"] = (suffer_df[col] - suffer_df[col].mean()) /
    ↪suffer_df[col].std()
suffer_df["combo_index"] = suffer_df["z_distance_km"] + suffer_df["z_max_hr"] +
    ↪suffer_df["z_elevation_m"]

predictors = [
    ("distance_km", "Distance (km)"),
    ("max_hr", "Max HR (bpm)"),
    ("elevation_m", "Elevation (m)"),
    ("combo_index", "Combo index (z-scored sum)"),
]

type_labels = {0: "Easy", 1: "Race", 2: "Long run", 3: "Workout/tempo"}
type_colors = {0: "#4A90D9", 1: "#E8593C", 2: "#9B59B6", 3: "#F39C12"}

fig, axes = plt.subplots(2, 2, figsize=(12, 9))
axes = axes.flatten()

for ax, (col, label) in zip(axes, predictors):
    x = suffer_df[col]
    y = suffer_df["suffer_score"]

    colors = suffer_df["workout_type"].fillna(0).astype(int).map(type_colors).
    ↪fillna("#4A90D9")
    ax.scatter(x, y, c=colors, alpha=0.8, edgecolors="none", s=60)

    mask = x.notna() & y.notna()
    z = np.polyfit(x[mask], y[mask], 1)
    r = np.corrcoef(x[mask], y[mask])[0, 1]
    xline = np.linspace(x.min(), x.max(), 100)
    ax.plot(xline, np.poly1d(z)(xline), color="#E8593C", linewidth=1.5,
    ↪linestyle="--")

    ax.set_xlabel(label)
    ax.set_ylabel("Relative Effort")
    ax.set_title(f"Relative Effort vs {label} (r = {r:.2f})")
    ax.spines[["top", "right"]].set_visible(False)

# Legend - only show types that appear in the data
```

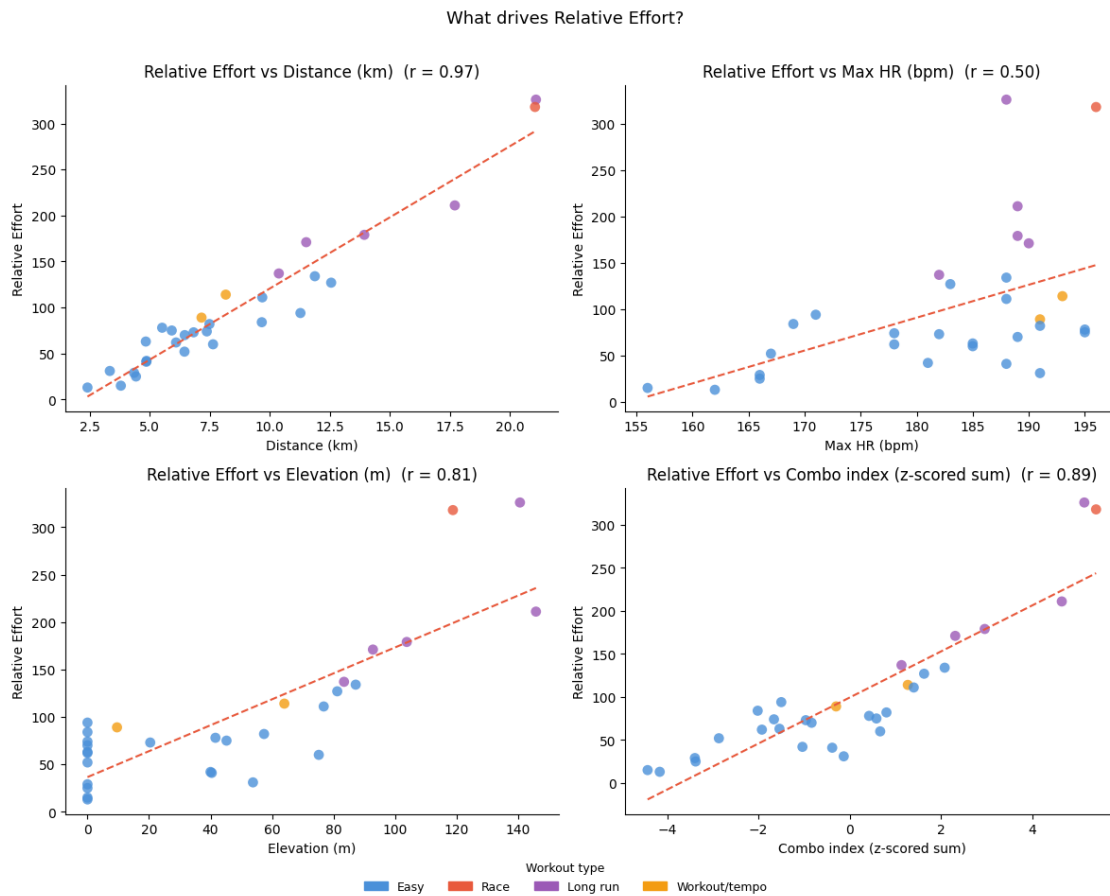


```

present_types = suffer_df["workout_type"].fillna(0).astype(int).unique()
legend_handles = [
    Patch(color=type_colors[t], label=type_labels[t])
    for t in sorted(present_types)
    if t in type_labels
]
fig.legend(
    handles=legend_handles,
    title="Workout type",
    loc="lower center",
    ncol=len(legend_handles),
    bbox_to_anchor=(0.5, -0.04),
    frameon=False,
    fontsize=9,
    title_fontsize=9,
)

plt.suptitle("What drives Relative Effort?", fontsize=13, y=1.01)
plt.tight_layout()
plt.gca()

```



```

[ ]: div = weekly.dropna(subset=["total_suffering"]).copy()

div["z_km"] = (div["total_km"] - div["total_km"].mean()) / div["total_km"].std()
div["z_suffering"] = (div["total_suffering"] - div["total_suffering"].mean()) /
    ↪div["total_suffering"].std()

# positive value = suffered more despite shorter distance
# negative value = suffered less despite longer distance
div["divergence"] = div["z_suffering"] - div["z_km"]

colors_ = ["#E8593C" if d > 0 else "#4A90D9" for d in div["divergence"]]

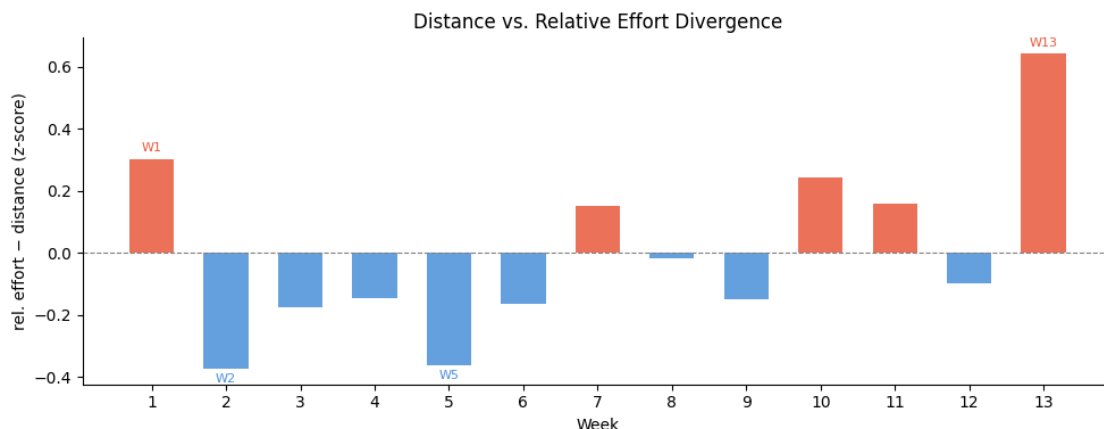
fig3, ax3 = plt.subplots(figsize=(10, 4))

bars = ax3.bar(div["week_num"], div["divergence"], color=colors_, alpha=0.85,
    ↪width=0.6)
ax3.axhline(0, color="var(--t)" if False else "grey", linewidth=0.8,
    ↪linestyle="--")

top2 = div.nlargest(2, "divergence")
bot2 = div.nsmallest(2, "divergence")
for _, row in pd.concat([top2, bot2]).iterrows():
    label_ = f"W{int(row['week_num'])}"
    yoff = 0.08 if row["divergence"] > 0 else -0.14
    ax3.annotate(label_, xy=(row["week_num"], row["divergence"]),
        xytext=(0, yoff * 60), textcoords="offset points",
        ha="center", fontsize=8,
        color="#E8593C" if row["divergence"] > 0 else "#4A90D9")

ax3.set_xlabel("Week")
ax3.set_ylabel("rel. effort - distance (z-score)")
ax3.set_title("Distance vs. Relative Effort Divergence")
ax3.xaxis.set_major_locator(ticker.MultipleLocator(1))
ax3.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.gca()

```



I'm somewhat surprised that Max HR and Suffer Score/Relative Effort has such a low correlation value, but it does make intuitive sense to me that total distance is the most significant contributor to those values.

Now, let's review the divergence between distance and Relative Effort. After reviewing my training plan, I can clearly identify causes for some of the highest positive differences: - **Week 1**: The first week of training is always the hardest! - **Week 7**: Lots of sprint work, including hill sprints and a brutal 1 minute sprint x 10 repeat workout. - **Weeks 10-11**: Increased tempo work before the taper began. - **Week 13**: Race day! I was suffering for sure...

And for the biggest negative differences: - **Week 2**: I was running near the beaches in Baja California Sur, Mexico. Hard to get stressed in that environment! - **Week 5**: Honestly, I was just hyped up about the progress being made in my fitness. A (private) Instagram story confirms this! :)

Sprints and tempo work have always been difficult for me. However, I know they are an integral part of building fitness, so I try my hardest to nail those workouts when they come up. The increased Relative Effort of that middle third of my training plan reflects the difficulty I faced with those sessions, but I truly believe they paid off on race day!

0.5 Long Runs and Pace

Long runs are [critical to any distance running training plan](#). They build endurance by increasing VO2 Max, help prevent injury by gradually increasing weekly load, and contribute to mental toughness. All of these are key factors in training, and the quality and frequency of your long runs can make or break a half marathon race.

Even though I was not perfectly consistent with my training plan, I made my best effort to *at least* complete my long run each week. My first long run was 10k, and I gradually increased distance as time passed. As mentioned earlier, long runs should make up $\leq 30\%$ of the total distance run per week.

I want to see how my average pace increased as a function of long runs completed, while examining the proportion of long run distance to total weekly distance. My average pace did improve over the

training period, as I observed my “easy” runs landed at about 1 min/mile (or about 0.62 min/km) faster than I began, once all was said and done.

```
[ ]: long_runs = df[df["distance_km"] >= 10].copy()
x_n = (df["date"] - df["date"].min()).dt.days.values

long_runs = long_runs.merge(
    weekly[["week_num", "total_km"]], on="week_num", how="left"
)
long_runs["pct_of_week"] = (long_runs["distance_km"] / long_runs["total_km"] * 100).round(1)

fig4, ax4 = plt.subplots(figsize=(10, 4))

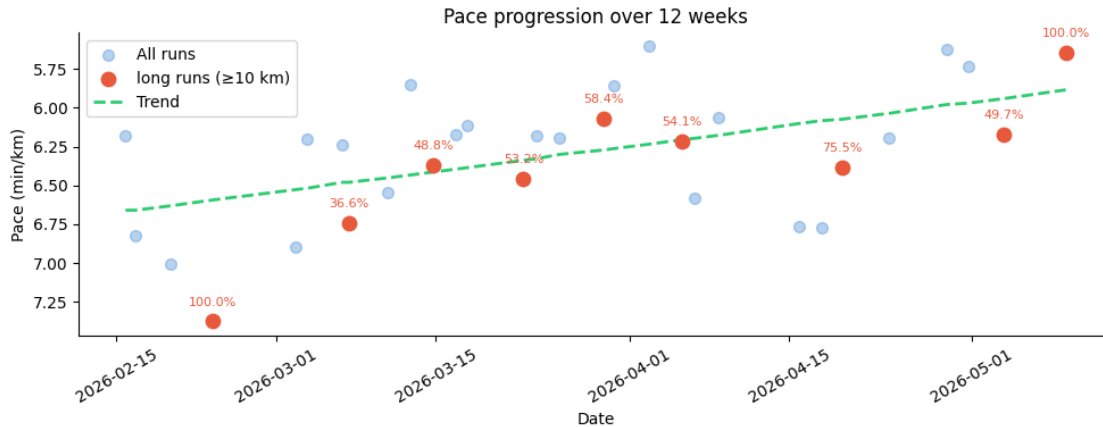
ax4.scatter(df["date"], df["pace_min_per_km"], alpha=0.4, color="#4A90D9", s=50, label="All runs")
ax4.scatter(long_runs["date"], long_runs["pace_min_per_km"], color="#E8593C", s=80, zorder=5, label="long runs ( 10 km)")
ax4.plot(df["date"], np.poly1d(np.polyfit(x_n, df["pace_min_per_km"].values, 1))(x_n), color="#2ecc71", linewidth=2, linestyle="--", label="Trend")

pct_dists = []

for _, row_ in long_runs.iterrows():
    ax4.annotate(
        f"{row_['pct_of_week']}%",
        xy=(row_["date"], row_["pace_min_per_km"]),
        xytext=(0, 10),
        textcoords="offset points",
        ha="center",
        fontsize=8,
        color="#E8593C",
    )
    pct_dists.append(row_['pct_of_week'])

mean_pct_dist = round(sum(pct_dists) / len(pct_dists), 2)

ax4.invert_yaxis()
ax4.set_title("Pace progression over 12 weeks")
ax4.set_xlabel("Date")
ax4.set_ylabel("Pace (min/km)")
ax4.legend()
ax4.spines[["top", "right"]].set_visible(False)
ax4.tick_params(axis="x", rotation=30)
plt.tight_layout()
plt.gca()
```



```
[ ]: mo.md(rf"""
Yikes.. it looks like my long runs (red points) made up an average of {mean_pct_dist}% of my weekly distance. While I did steadily build my total
distance and pace over the course of the training plan, I should definitely
up my mid-week mileage next time to avoid overtraining on weekends.
""")
```

Yikes.. it looks like my long runs (red points) made up an average of 64.03% of my weekly distance. While I did steadily build my total distance and pace over the course of the training plan, I should definitely up my mid-week mileage next time to avoid overtraining on weekends.

0.6 Race Day Deep-Dive

Now that I've taken a look at the quality of my training, it's time to analyze the culmination of it all: race day!

On May 9, 2026, at 6 AM PST, my fiancé and I lined up at the top of a pine-blanketed hill in Bass Lake, CA for our race. The gun was fired, and we were off...

Let's take a look at my mile splits. To do this, I'll need to get the Streams, a more detailed view of an activity from Strava, from my race. Since there should only be one race in this time period, I'm not going to implement persistence logic for this set.

```
[ ]: def fetch_streams(activity_id, access_token):
    resp = requests.get(
        f"https://www.strava.com/api/v3/activities/{activity_id}/streams",
        headers={"Authorization": f"Bearer {access_token}"},
        params={"keys": "time,distance,heartrate,velocity_smooth,altitude",
        ↪ "key_by_type": "true"},
    )
    resp.raise_for_status()
    return resp.json()
```

```

streams = None
race_id = [a for a in raw_runs if a.get("workout_type") == 1][0]["id"]
race_name = [a for a in raw_runs if a.get("workout_type") == 1][0]["name"]

if race_id:
    streams = fetch_streams(race_id, ACCESS_TOKEN)
    print(f"fetch streams for {race_name}")

```

fetch streams for Yosemite Half Marathon

```

[ ]: if not streams:
    mo.stop(True, mo.callout(mo.md("Race ID is required to see the race_
    ↳breakdown."), kind="warn"))

dist = streams["distance"]["data"]
hr = streams.get("heartrate", {}).get("data")
vel = streams["velocity_smooth"]["data"]
alt = streams.get("altitude", {}).get("data")

race_df = pd.DataFrame({
    "dist_km": [d / 1000 for d in dist],
    "pace": [1 / (v * 60 / 1000) if v > 0 else None for v in vel],
    "hr": hr,
    "alt": alt,
})
race_df["km_bin"] = race_df["dist_km"].apply(lambda x: int(x))

splits = race_df.groupby("km_bin").agg(
    avg_pace=("pace", "mean"),
    avg_hr=("hr", "mean") if hr else ("dist_km", "count"),
).reset_index()

# km to mile conversion: 1 mile = 1.60934 km
splits["avg_pace_mile"] = splits["avg_pace"] * 1.60934

n_plots = 4 if (hr and alt) else (3 if (hr or alt) else 2)
fig5, axes5 = plt.subplots(n_plots, 1, figsize=(11, 3 * n_plots), sharex=True)
if n_plots == 1:
    axes5 = [axes5]

ax_idx = 0
mean_pace = splits["avg_pace"].mean()

# pace in km
axes5[ax_idx].bar(splits["km_bin"], splits["avg_pace"], color="#4A90D9",
    ↳alpha=0.8, width=0.7)

```

```

axes5[ax_idx].axhline(mean_pace, color="#E8593C", linestyle="--", linewidth=1.
    ↪5, label=f"Avg {mean_pace:.2f} min/km")
axes5[ax_idx].invert_yaxis()
axes5[ax_idx].set_ylabel("Pace (min/km)")
axes5[ax_idx].set_title("Race Day Splits")
axes5[ax_idx].legend(fontsize=8)
axes5[ax_idx].spines[["top", "right"]].set_visible(False)
ax_idx += 1

# pace in mi
mean_pace_mile = splits["avg_pace_mile"].mean()

def fmt_pace(minutes):
    m = int(minutes)
    s = int(round((minutes - m) * 60))
    return f"{m}:{s:02d}"

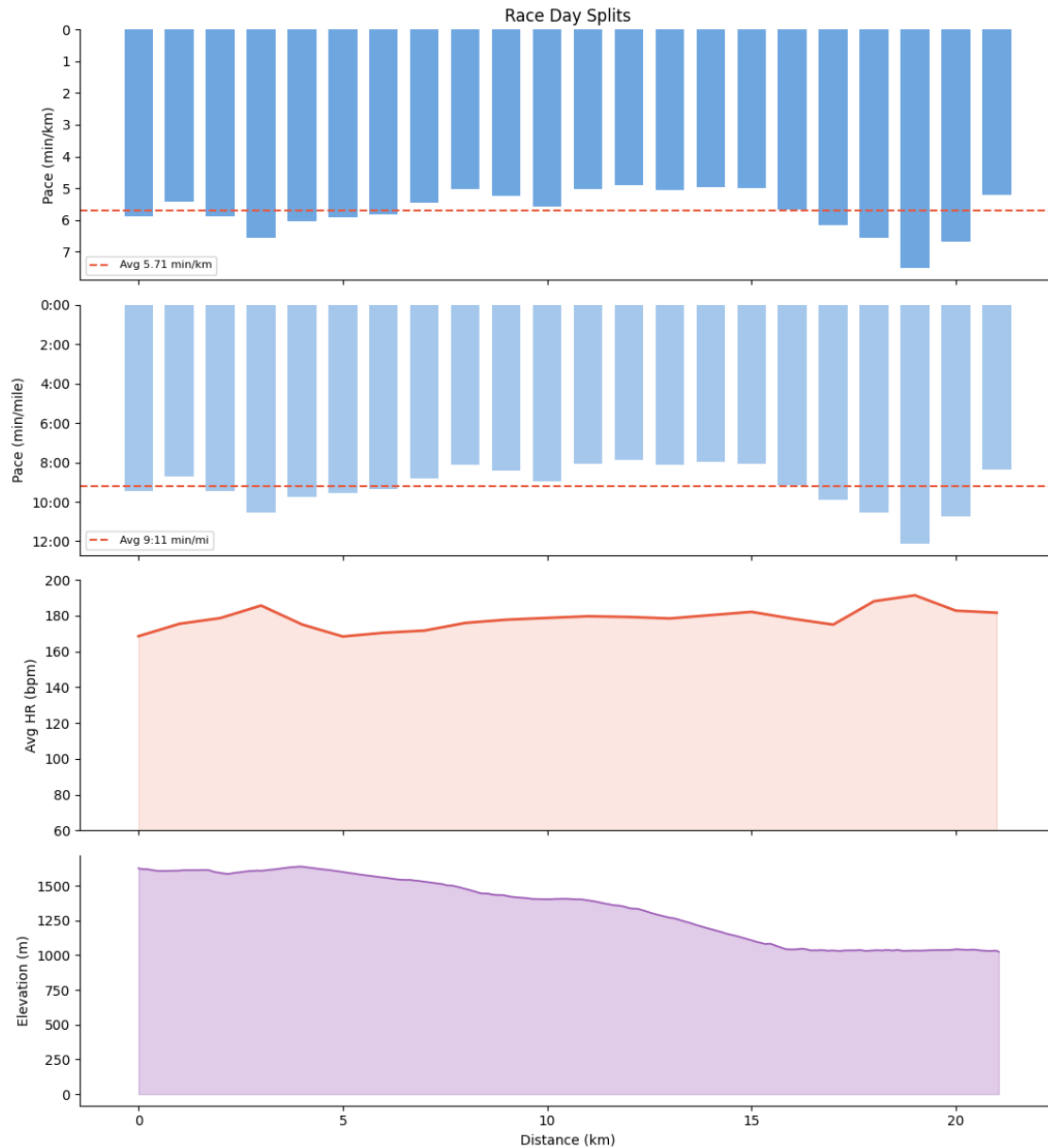
axes5[ax_idx].bar(splits["km_bin"], splits["avg_pace_mile"], color="#4A90D9",
    ↪alpha=0.5, width=0.7)
axes5[ax_idx].axhline(mean_pace_mile, color="#E8593C", linestyle="--",
    ↪linewidth=1.5, label=f"Avg {fmt_pace(mean_pace_mile)} min/mi")
axes5[ax_idx].invert_yaxis()
axes5[ax_idx].set_ylabel("Pace (min/mile)")
axes5[ax_idx].yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _:
    ↪fmt_pace(x)))
axes5[ax_idx].legend(fontsize=8)
axes5[ax_idx].spines[["top", "right"]].set_visible(False)
ax_idx += 1

if hr:
    axes5[ax_idx].plot(splits["km_bin"], splits["avg_hr"], color="#E8593C",
    ↪linewidth=2)
    axes5[ax_idx].fill_between(splits["km_bin"], splits["avg_hr"], alpha=0.15,
    ↪color="#E8593C")
    axes5[ax_idx].set_ylabel("Avg HR (bpm)")
    axes5[ax_idx].set_ylim(60, 200)
    axes5[ax_idx].spines[["top", "right"]].set_visible(False)
    ax_idx += 1

if alt:
    axes5[ax_idx].fill_between(race_df["dist_km"], race_df["alt"], alpha=0.3,
    ↪color="#9B59B6")
    axes5[ax_idx].plot(race_df["dist_km"], race_df["alt"], color="#9B59B6",
    ↪linewidth=1.2)
    axes5[ax_idx].set_ylabel("Elevation (m)")
    axes5[ax_idx].spines[["top", "right"]].set_visible(False)

```

```
axes5[-1].set_xlabel("Distance (km)")
plt.tight_layout()
plt.gca()
```



0.7 Pace Variation: Grade and Fatigue

As you can see from the above chart, there was some significant variation in my pace throughout the race. Some of this was definitively due to the altitude variation along the course: the first 3-4 miles of the course showcased beautifully brutal rolling hills through wooded fire roads. After

that came my favorite part of the course: 7 miles of sweet, paved, downhill bliss. You can see that the pace dropped dramatically, by almost 1 min/km or 2 min/mi below average. The finish line emerged after 3 miles of “gentle” terrain around Bass Lake, within which I successfully avoided passing out.

One of my favorite features in the Strava app is the Grade Adjusted Pace data. I train in a hilly area, so it’s always valuable for me to see how much a long climb *really* slows me down. Some [online discourse](#) has gone on about the effectiveness of Strava’s proprietary model versus traditional methods, developed by [Minetti et al. in 2002](#). My Strava indicates that my race day Grade Adjusted Pace is slightly slower (~10 s/mi, at 9:26 min/mi) than my raw pace. Let’s contribute to the discourse and compare Strava’s output with the Minetti model.

```
[ ]: # gradient correction: ~1 min/km per 100m/km grade (Minetti et al.)
race_df["gradient"] = race_df["alt"].diff() / (race_df["dist_km"].diff() *
    ↪1000) * 100
race_df["gradient"] = race_df["gradient"].clip(-20, 20)
race_df["pace_adjusted"] = race_df["pace"] - (race_df["gradient"] * 0.01)

adj_splits = race_df.groupby("km_bin").agg(
    avg_pace=("pace", "mean"),
    avg_pace_adj=("pace_adjusted", "mean"),
    avg_gradient=("gradient", "mean"),
).reset_index()

fig_adj, ax_adj = plt.subplots(figsize=(11, 4))

x_adj = adj_splits["km_bin"]
width = 0.35

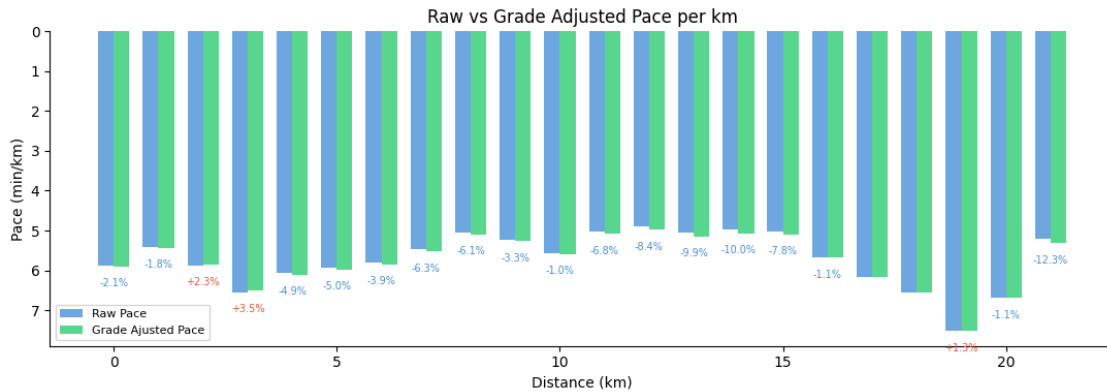
ax_adj.bar(x_adj - width/2, adj_splits["avg_pace"], width, color="#4A90D9",
    ↪alpha=0.8, label="Raw Pace")
ax_adj.bar(x_adj + width/2, adj_splits["avg_pace_adj"], width, color="#2ecc71",
    ↪alpha=0.8, label="Grade Adjusted Pace")

ax_adj.invert_yaxis()
ax_adj.set_ylabel("Pace (min/km)")
ax_adj.set_xlabel("Distance (km)")
ax_adj.set_title("Raw vs Grade Adjusted Pace per km")
ax_adj.legend(fontsize=8)
ax_adj.spines[["top", "right"]].set_visible(False)

for _, row_adj in adj_splits.iterrows():
    if abs(row_adj["avg_gradient"]) > 1:
        label_grad = f"{row_adj['avg_gradient']:.1f}%"
        color = "#E8593C" if row_adj["avg_gradient"] > 0 else "#4A90D9"
        y_pos = max(row_adj["avg_pace"], row_adj["avg_pace_adj"])
        ax_adj.annotate(label_grad, xy=(row_adj["km_bin"], y_pos),
            xytext=(0, -14), textcoords="offset points",
```

```
ha="center", fontsize=7, color=color)
```

```
plt.tight_layout()
plt.gca()
```



Looking at these tightly-coupled bars, Strava and Minetti seem to agree: the grade of the course didn't affect my pace *that* much. Though I'd like to blame my slow finish on a lack of downhill terrain, the data suggests that I was simply tired. Accounting for the fact that I missed my first feed window (30 min into the race), it makes sense that my body had run out of energy past mile 10.

Since I've determined that the course's grade made little difference in my pace overall, I want to take a closer look at my pace variation throughout the race.

```
[ ]: total_dist = race_df["dist_km"].max()
      halfway   = total_dist / 2

race_df["half"] = race_df["dist_km"].apply(lambda x: "First half" if x <=
      ↪halfway else "Second half")

half_avg  = race_df.groupby("half")["pace"].mean()
overall   = splits["avg_pace"].mean()

splits["pace_delta"] = splits["avg_pace"] - overall
splits["half"] = splits["km_bin"].apply(
    lambda k: "First half" if (k + 0.5) <= halfway else "Second half"
)

fig_split, (ax_top, ax_bot) = plt.subplots(2, 1, figsize=(11, 7))

colors_split = ["#E8593C" if d > 0 else "#2ecc71" for d in splits["pace_delta"]]
ax_top.bar(splits["km_bin"], splits["pace_delta"], color=colors_split, alpha=0.
    ↪85, width=0.7)
```

```

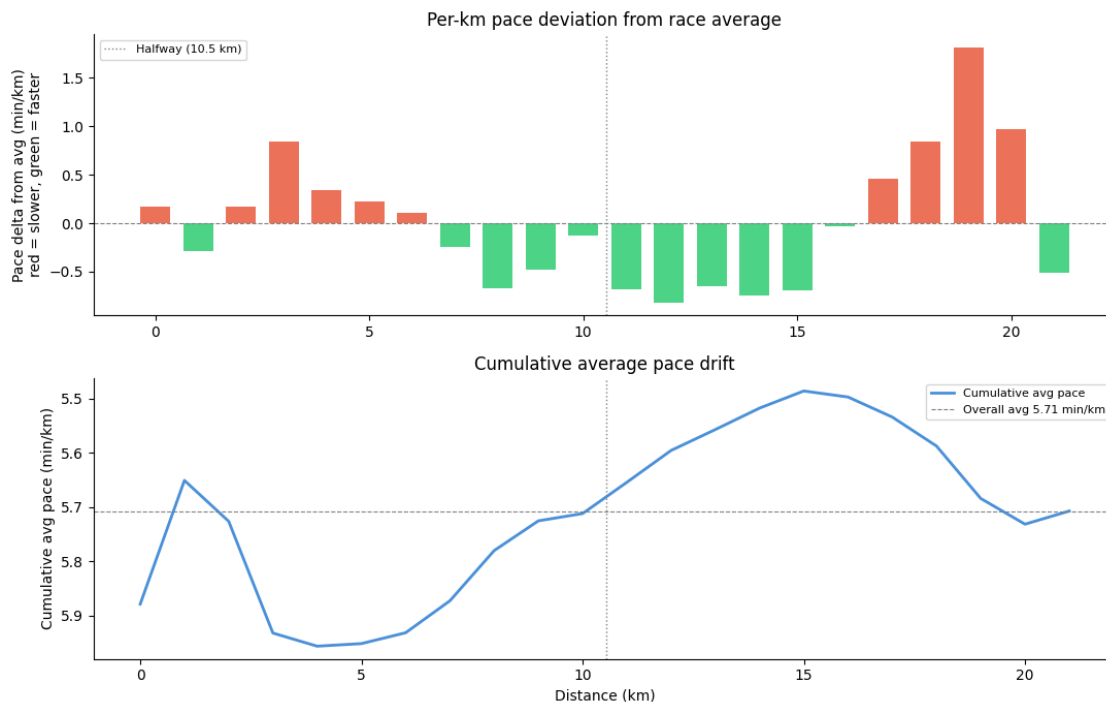
ax_top.axhline(0, color="grey", linewidth=0.8, linestyle="--")
ax_top.axvline(halfway, color="grey", linewidth=1, linestyle=":",
    ↳label=f"Halfway ({halfway:.1f} km)")
ax_top.set_ylabel("Pace delta from avg (min/km)\nred = slower, green = faster")
ax_top.set_title("Per-km pace deviation from race average")
ax_top.legend(fontsize=8)
ax_top.spines[["top", "right"]].set_visible(False)

for half, grp in splits.groupby("half"):
    mid_km = grp["km_bin"].median()
    avg_d = grp["pace_delta"].mean()

splits["cum_avg_pace"] = splits["avg_pace"].expanding().mean()
ax_bot.plot(splits["km_bin"], splits["cum_avg_pace"], color="#4A90D9",
    ↳linewidth=2, label="Cumulative avg pace")
ax_bot.axhline(overall, color="grey", linewidth=0.8, linestyle="--",
    ↳label=f"Overall avg {overall:.2f} min/km")
ax_bot.axvline(halfway, color="grey", linewidth=1, linestyle=":")
ax_bot.invert_yaxis()
ax_bot.set_ylabel("Cumulative avg pace (min/km)")
ax_bot.set_xlabel("Distance (km)")
ax_bot.set_title("Cumulative average pace drift")
ax_bot.legend(fontsize=8)
ax_bot.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.gca()

```



It's evident from the deviation graph that the downhill section helped me run faster. Even if the slope itself wasn't extreme enough to affect my pace biomechanically, it would be foolish to say that a combination of gravity and cognitive relief couldn't give me a "boost" to negative-split the middle of the race. Similarly, it's clear that the view of the finish line in that last kilometer significantly aided my spirits (and thus, legs).

0.8 Conclusion

I've learned that consistency is king in distance sports. Though I'm an amateur runner, I can feel the effects of a skipped workout keenly. This training set was no different— the weeks where I missed out on mileage affected my subsequent Relative Effort, and I may not have burned out at mile 10 had I hit those workouts on time. Whatever distance event I attempt next (looking at you, Olympic triathlon...), I hope to increase my dedication to the sport, hopefully bolstering my fitness and mental toughness along the way. Until next time!